

Module_5_report

1. How to fresh install/run via pip and uv

1a. Using pip

python -m venv venv source venv/bin/activate pip install -r src/requirements.txt pip install -e . python src/app.py

1b. Using uv

uv venv source venv/bin/activate uv pip sync src/requirements.txt uv pip install -e . python src/app.py

pip install -r src/requirements.txt installs the required Python packages. pip install -e . installs the project in editable mode. uv pip sync src/requirements.txt makes the environment match the requirements exactly.

2. Dependency graph summary.

The dependency graph shows that app.py is the main entry point for the Flask web application.

There is an arrow from flask->app.py which means app.py depends on Flask. It depends on Flask to render pages, handle requests, redirect users, and return JSON responses. App.py depends on psycopg to connect to the PostgreSQL database and execute sql queries.

queries_2â†’app.py means app.py depends on queries from queries_2.py, which has the parameterized pre-defined queries used by the dashboard.

The large clusters around flask and psycopg are internal modules those packages depend on. psycopg_sql->queries_2 means queries_2 depends on psycopg_sql to parameterize the queries. In queries_2 it leverages sql.SQL and sql.Identifier to parameterize the queries.

3. My SQL injection defenses (what changed and why itâ€™s safe)

My SQL injection defenses involve changing the code so user inputs are no longer inserted directly into SQL strings. I used psycopg.sql.SQL for the SQL statement structure, which leverages sql.Identifier() to quote the table and column names. In this case user inputs are not treated as executable sql queries. Each query also includes a LIMIT %s with clamp_limit(), which prevents accidental or malicious requests from returning too many rows.

4. Least-privilege DB configuration (what permissions and why)

I removed hard-coded database credentials from the app code and moved the connection settings into environment variables such as DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASSWORD. The values of these environment variables are stored inside .env file, and I listed .env in .gitignore so that I wonâ€™t push these secret values into remote repo.

I created a new application user, gradcafe_app_user. This user is not a superuser so it can not perform dangerous actions such as DROP or ALTER. This user can perform necessary actions like select and insert, which is required by pull-data function on the web.

5. LIMIT enforcement

I have refactored the queries so that each query includes a LIMIT %s with clamp_limit(), which prevents accidental or malicious requests from returning too many rows.

6. CI workflow

The GitHub CI workflow runs automatically on every push and pull request. It has four separate jobs: Pylint, dependency graph generation, Snyk dependency scanning, and Pytest. It runs pylint and fails if score is below 10.

The dependency graph job generates dependency.svg from app.py and it fails if the graph is not created.

The Snyk job installs the Snyk CLI and runs snyk test against src/requirements.txt to scan dependencies for known vulnerabilities.

The Pytest job installs all the required packages, then runs the whole test suite.